

Q:1:- Performance testing:

- learning the fundamentals of performance testing?

Ans:- Performance Testing is considered to be among the most essential and important [types of software testing](#). It's the go-to method to understand what would happen if an application is put under extreme conditions. Thanks to performance testing, testers can define appropriate corrective actions to ensure the app's peak performance before it is launched to the market.

What is Performance Testing?

As a non-function technique of [software testing in financial services](#), performance testing helps to evaluate an application's performance metrics (stability, speed, scalability, and responsiveness) under a given workload. In other words, it seeks to load an application with simulated traffic or users to detect performance bottlenecks. Back then, Performance Testing used to be an addition to the software development process before it became one of the most basic and essential testing steps.

Performance Testing addresses the following questions:

1. Is the application able to handle a given number of concurrent users?
2. Is the average response time for pages acceptable under a given load?
3. Will the application return to normal behaviour after experiencing a load peak?
4. How many concurrent users can the application handle while maintaining an adequate response time?
5. What is the threshold of load that is above it? The application starts to make errors and/or reject connections.
6. Under high load, does the application remain functional, or does it crash?

Types of Performance Tests

There are various types of performance testing; but in general, Load Testing, Unit Testing, Stress Testing, Soak Testing, and Spike Testing are the most popular ones.

Load Testing

Load testing is performed to simulate how the application behaves when it is loaded with increasing traffic or an increasing number of concurrent users. For example, a load can be a specific number of users logging in to a banking application to make a specific number of transactions within a specific duration. In this case, performance testing aims to find out the response time of all the involved functions to identify bottlenecks. Though load testing is perhaps the simplest form of performance testing, it's the most useful when evaluating system behaviour, latency, bottlenecks, and throughput.

Stress Testing

A stress test is technically a load test, but the load exceeds the limits of the application under the test. The purpose of stress testing is to observe how the app behaves under such extreme conditions. Stress testing helps you know how and when your application crashes. One typical stress test is gradually increasing the load to see where and when the application violates non-functional requirements. This point at which the app collapses can be used to represent its capacity. For example, "the app can handle a total of 976 concurrent users when it starts to violate the requirements".

Spike Testing

If you want to see how your app responds to sudden bursts of traffic, spike testing is the way. Spike Testing is very much similar to Load or Stress Testing, in which large traffic or users are loaded to the app; but a spike test comes in the form of a sudden spike in traffic. Typically, Spike Testing is used to see if the app's performance might plummet or its functionalities collapse when put under sudden spikes, or if it can handle such extreme conditions. Other than that, it can be used to prepare for upcoming marketing or promotional events, where there would likely be a traffic burst to the app under test. Spike Testing is a must for applications that, from time to time, experience unexpected traffic increases.

Soak Testing

As another type of load testing, a soak test (also known as endurance test) still delivers an increasing load (i.e. increasing traffic or number of concurrent users) to the app, but for a longer period of time. The purpose is to see if such

sustained intense activity would cause any performance drops or consequences, such as leakages of memory or disk space filling up. While it depends, the duration of a typical soak test is normally run for hours.

Scalability Testing

Scalability seeks to evaluate the application's ability to scale up or scale down. A typical approach is to execute a series of stress tests, observe, and then gather data on how the app can scale. Attributes to measure include response time, throughput, hits & requests per second, transaction processing speed, CPU usage, and more. The results of scalability testing are usually used in planning and designing software development, which helps to reduce the cost and possibility of issues related to performance.

Factors that Performance Testing tries to measure

Thanks to performance testing, various factors critical to a successful application, such as response times and potential errors, are evaluated. You can then leverage the results to identify bottlenecks, performance issues, and bugs so that you can best plan how to resolve those problems. In general, performance issues are related to speed, response times, load times, and scalability.

Load time: the time it takes to start an application. Delays should be as short as possible or be avoided altogether.

Response time: the time elapses between the moment users finish performing an action (for example, entering information into the app) and the moment the app responds to that action.

Scalability: the application's ability to handle a growing workload or number of users. Poor scalability poses a major problem; for example, the app can work well with users equal to or fewer than 5000, any number bigger than this would disrupt it.

Bottlenecks: any inefficiency that obstructs the applications, thus causing performance issues. Bottlenecks are typically the results of poorly written code or hardware issues.

The performance testing process

While a performance testing process can vary depending on the nature of the application under test, you can refer to the following 7 steps as the basis.

Step 1: Identify the test environment: By first identifying the tools (both hardware and software) and network needed for performance testing, you will be able to create more effective tests, and also anticipate possible challenges that may happen during the process

Step 2: Identify performance acceptance criteria: Performance acceptance criteria typically are the goals and constraints for throughput, response times, and resource allocation. Normally, these criteria are set by business analysts only, but testers should also be involved in it as they are better at the technicalities of the app. But if you are unable to define any criteria, you can refer to other similar apps as the benchmark

Step 3: Plan & Design Performance Tests: The team assesses the variance of usage among different types of end-users and identifies possible scenarios to test all those use cases. It is important to simulate as many end-users as possible, design data for performance tests, and define key metrics.

Step 4: Configuring the test environment: The test environment is prepared for performance tests before they are executed. In addition, the related tools and other resources for the test run are arranged.

Step 5: Implement test design: All the performance tests are created as per the test design

Step 6: Run the tests: Performance tests are executed and monitored

Step 7: Analyse, adjust, and retest: Test results of the previous step are consolidated, analysed, and shared across stakeholders. Then the testing team would adjust the tests and run them again to see if there would be any boost or drop in performance. Generally, improvements between each retest would decrease; therefore, you should stop when bottlenecks are deemed to be caused by the CPU. This is when you consider increasing CPU power.

Metrics for Performance Testing

Metrics are important to measure your performance testing efforts. These parameters generally include:

1. The total amount of time the processor executes non-idle threads

2. Usage of the computer's physical memory for the processing
3. Bandwidth, or the number of bits per second consumed by the network interface
4. The amount of time the disk is busy with reading or write requests
5. The usage of virtual memory
6. The number of pages that are written or read to disk for resolving hardware page defects
7. Rate of processing of faulty pages by the processor
8. Number of hardware interruptions the processor receives per second
9. Average read and write requests that are queued for the chosen disk during a sampling interval
- 10.Length of the output packet queue
- 11.Total number of bytes sent and received by the interface per second
- 12.Amount of response time
- 13.The rate at which a computer or network receives requests per second
- 14.Number of user requests met by pooled connections
- 15.Maximum number of concurrently active sessions
- 16.Number of SQL statements handled by cached data instead of I/O devices
- 17.Number of access requests to a file on a web server per second
- 18.The data that can be restored
- 19.The tables' and databases' locking quality
- 20.Maximum wait times
- 21.Number of threads that are currently running or active
- 22.The rate of return of unused memory

Conclusion

Performance Testing is key to an application before it's released to the market. Therefore, you want to make sure that your app's performance is tested before launching. While it's true that no amount of testing is enough to ensure perfect performance, costly flaws and mistakes can be prevented by well-planned performance tests. Today's businesses also apply [Test Automation](#) to enhance the effectiveness of their Performance Testing.